Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I will definitely need loops and conditionals, as I think I will have my program append user answers to a list, and then go through the list at the end and list the user's answers. Lists and classes are required for the project. I will have a State dataclass, and this will have a list of MathProblem dataclass, which will provide the history of the math problems answered. I want to get better at the planning of a program, as right now I feel kind of overwhelmed trying to think of what my project will need.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I think that writing the functions may be the easiest, because I have had a lot of practice with writing functions. However, the parts that involve designing the pages of the application will be the hardest. I have not had much practice with Drafter when compared to other things such as conditionals, loops, lists, etc., so I think these parts will be the hardest.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

I estimate that designing the pages where the users complete the math problems will be the part that requires the most time and effort. The main page of the application should be implemented first with the buttons that go to the route functions, and then the route functions should be implemented.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

All of my other schoolwork and projects may become distractions that interfere with my work on this project. All of my deadlines are overlapping at the moment, causing me to become quite overwhelmed and distracted by all the work I need to complete. I will try to take it one day at a time and properly split my time so I ensure I have time for everything.

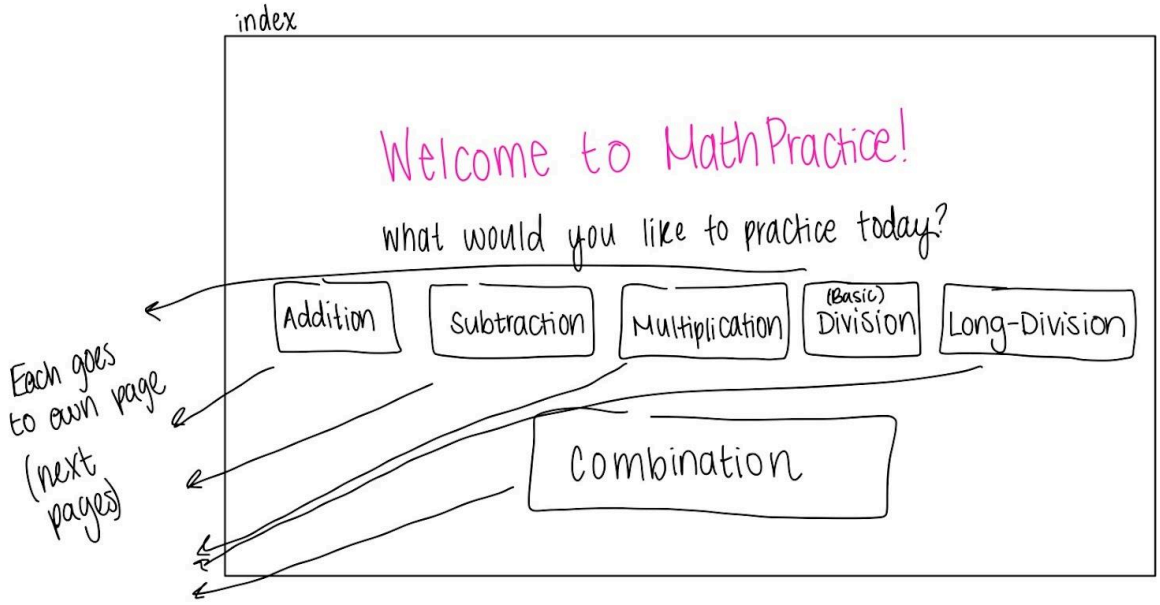
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.



State:

- current_page: str
- first_number: int
- second_number: int
- operation: str
- correct_answer: int

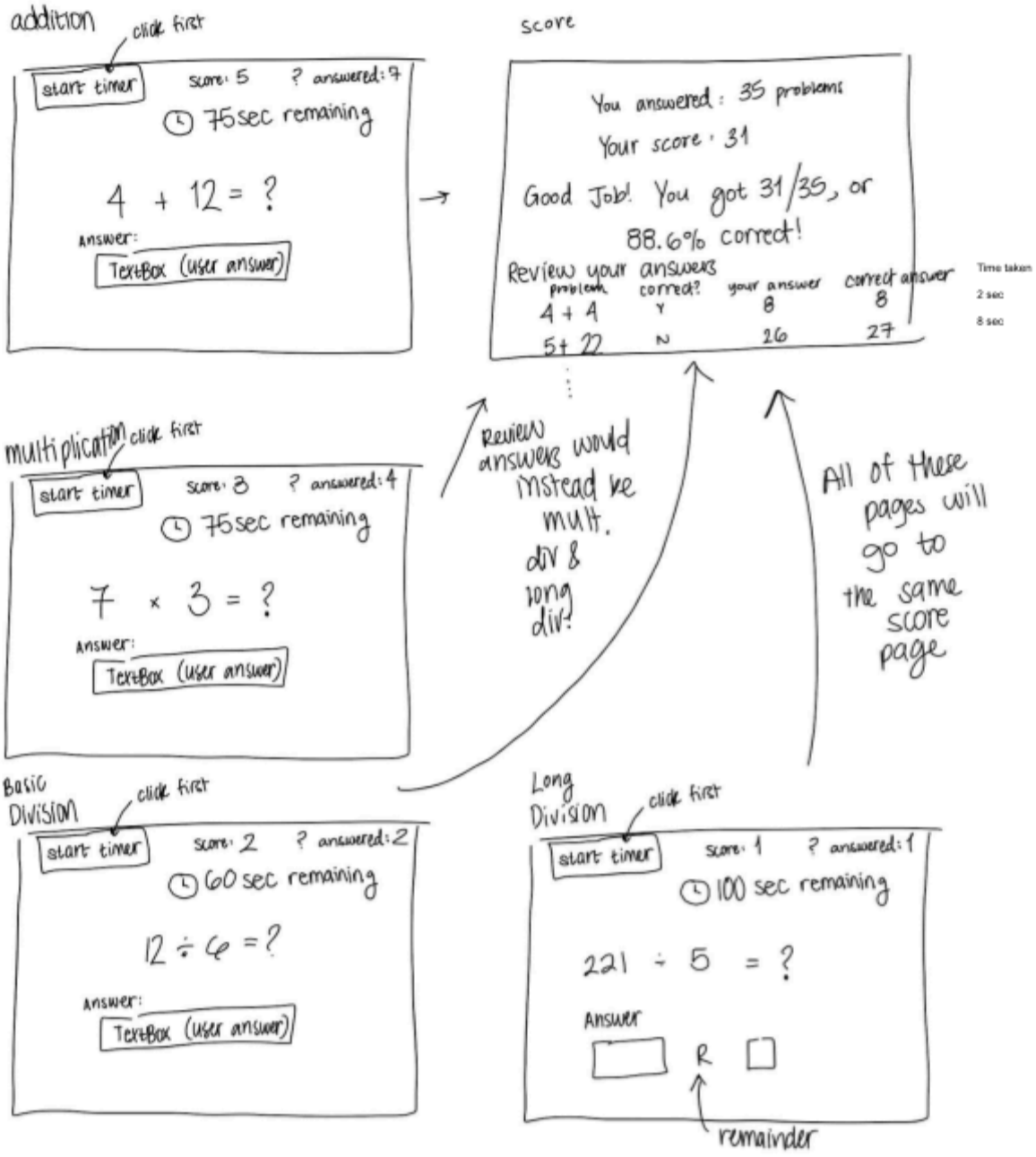
(for current problem)

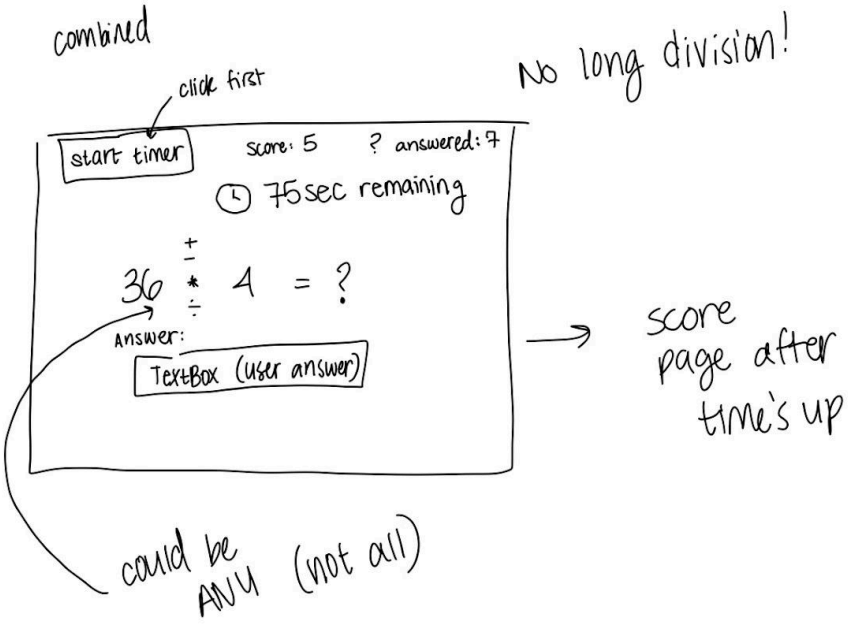
- user_answer: str
- feedback: str
- score: int
- questions_answered: int
- problem_history: list [Math Problem]
- time_limit: int
- time_remaining: int
- timer_running: bool

Math Problem

- first_number: int
- second_number: int
- operation: str
- correct_answer: int
- user_answer: str
- was_correct: bool
- time_taken: int

*This class will store all math problems the user has answered so they can review their answers after time is up


addition
click first
start timer
score: 5
? answered: 7
75 sec remaining
4 + 12 = ?
Answer:
TextBox (user answer)
score
You answered: 35 problems
Your score: 31
Good Job! You got 31/35, or
88.6% correct!
Review your answers
problem
correct?
your answer
correct answer
Time taken
4 + 4
Y
8
8
2 sec
5 + 22
N
26
27
8 sec
multiplication
click first
start timer
score: 3
? answered: 4
75 sec remaining
7 × 3 = ?
Answer:
TextBox (user answer)
Review answers would instead be mult. div & long div!
All of these pages will go to the same score page
Basic Division
click first
start timer
score: 2
? answered: 2
60 sec remaining
12 ÷ 6 = ?
Answer:
TextBox (user answer)
Long Division
click first
start timer
score: 1
? answered: 1
100 sec remaining
221 ÷ 5 = ?
Answer
R
remainder


combined
click first
No long division!
start timer
score: 5
? answered: 7
75sec remaining
36
+
−
*
÷
4
= ?
answer:
TextBox (user answer)
score page after time's up
could be ANY (not all)

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**The user will be able to practice the basic math operation of their choice for the time duration of their choice. They will then receive a score and a list of their answers so they can practice the ones they got incorrect.**
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	current_page: str (this will update when user chooses what operation or when we go to the score page) first_number : int second_number: int operation: str correct_answer: int user_answer: str feedback: str (the attributes 2-7 will update with each question that is posed to the user) score:int (as the user gets correct answers, the score will increase by 1. Incorrect answers do not affect the score) questions_answered: int (will increase by one after every question is answered, correct or not) problem_history: list[MathProblem] (new problems will be appended to the list each time the user answers, described below) time_limit: int (chosen by the user when they reach their desired operation page. They will input in a textbox that will be in the same spot as the countdown in the mock-up) time_remaining: int (will decrease by one for every second that passes) timer_running: bool (will become true after the user

	clicks “Start timer” button)
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	MathProblem: first_number : int second_number: int operation: str correct_answer: int user_answer: str was_correct: bool time_taken:int None of these will be modified, as every problem will be recorded in this way and stored for the user.
Routes (minimum 7) The names of your route functions	index addition subtraction multiplication basic_division long_division combination score start_timer submit_answer
Pages (minimum 5) A brief description of the pages in your site	● The welcome page, which asks users to pick which operation they want to work with. ● The addition page, where users will answer addition problems. ● The subtraction page, where users will answer subtraction problems. ● The multiplication page, where users will answer multiplication problems. ● The basic division page, where users will answer basic division problems. ● The long division page, where users will answer long division problems. There will be an additional text box for the remainder (if there is any)

	• The combination page, where users will answer a combination of addition, multiplication, subtraction, and basic division problems. ○ All of the above pages will look almost identical, showing the timer, score, problems answered, current question, textbox, and submit_answer button • The score page, where users will see their score, how many questions they answered, and their answer history.
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	The user will input their answers for each problem they solve in a textbox. They will input as many times as they can in the selected time limit.
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	If the answer is correct: state.score increases by one. If the answer is submitted: state.questions_answered increases by one. If the user clicks “start timer”: state.timer_running = True If the user selects (input operation): state.operation = “(selected operation)”
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	The loop will loop over the list of MathProblem to sort by correct and incorrect answers (looking at the was_correct attribute), printing them separately to the user on the score page. All of the attributes will be printed in a row.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Chat Link (because copy/pasting is a little messy):
https://chatgpt.com/share/6aece6d3-a9e4-83ea-9125-1a20f5992f44

I am working on a drafter application that will allow users to practice their basic math skills. There will be different pages for addition, subtraction, multiplication, division, long division, and a combination. I am having a hard time envisioning my state dataclass, can you help me

Absolutely! For your math practice application, I would recommend using one main State dataclass that manages the current math problem, the user's answer, and their progress.

The key is to avoid creating a completely separate state for addition, subtraction, multiplication, division, and long division. Since these pages share a lot of functionality, you can reuse the same State dataclass across all of them.

Let's break it down.

1. Think about what your application needs to remember

Your application needs to keep track of a few things:

Information	Why you need it
Current page	Know which math operation the user is practicing
First number	Store the first number in the problem
Second number	Store the second number in the problem

Correct answer	Know what answer the user should get
User answer	Store the answer the user enters
Score	Track how many questions they get correct
Questions answered	Track how many problems they've attempted
Feedback	Tell the user whether they got the answer right or wrong

These are the main pieces of information I would start with.

2. A possible State dataclass

Here is what I envision for your application:

```
None
@dataclassclass State:    # Navigation    current_page: str
# Current math problem    number1: int    number2: int
operation: str    correct_answer: int    # User interaction
user_answer: str    feedback: str    # Progress tracking
score: int    questions_answered: int
```

Notice that we have an `operation` variable.

This is particularly useful because you don't need separate variables like:

- `addition_problem`
- `subtraction_problem`
- `multiplication_problem`
- `division_problem`

Instead, your application can use the same two numbers and change the operation.

For example:

number 1	operation	number 2	correct_answer
8	+	4	12
8	-	4	4
8	*	4	32
8	/	4	2

3. How the different pages would work

I would organize your application something like this:

Math Practice Application

Home Page

Choose a math activity

+

Addition

−

Subtraction

✖

Multiplication

➗

Division

➗

Long Division

±

Combination

Shared Practice Functionality

Generate problem → Accept answer → Check answer → Update score

Each page can update the `current_page` and `operation` variables.

For example:

- Addition page: `current_page = "addition"` and `operation = "+"`
- Subtraction page: `current_page = "subtraction"` and `operation = "-"`
- Multiplication page: `current_page = "multiplication"` and `operation = "*"`
- Division page: `current_page = "division"` and `operation = "/"`

Your problem-generation and answer-checking logic can then be reused.

4. What about long division?

Long division is the one page where I would consider adding a few extra State variables.

Unlike regular division, long division involves multiple steps.

For example, you might eventually want to track:

- The current step of the long division problem.
- The user's current intermediate answer.
- Whether the current step is correct.

You could add:

```
None
# Long divisionquotient: intremainder: intcurrent_step: int
```

However, I wouldn't add these until you are ready to implement long division.

You could also start with a simpler long division activity where the user enters the final quotient and remainder.

5. My recommendation: Start small!

Since this is an introductory programming project, I would avoid making your State dataclass overly complicated.

I would start with these variables:

Recommended starting point

```
None
@dataclassclass State:    current_page: str    number1: int
number2: int    operation: str    correct_answer: int
user_answer: str    feedback: str    score: int
questions_answered: int
```

You can always expand it later.

One important design decision: You do not need to store information that you can easily calculate. For example, you don't necessarily need to store the percentage of correct answers because you can calculate it using `score / questions_answered`.

6. One question about your application

I want to clarify one thing because it could influence how we design your State dataclass.

For the combination page, are you envisioning:

How should combination practice work?

Random operations: Each new question randomly chooses addition, subtraction, multiplication, or division.

Mixed worksheet: The user receives a set of questions containing all four operations.

User choice: The user chooses which operations to include.

How should regular division work?

Only division problems with whole-number answers.

Division problems that can have remainders.

A mixture of both.

Submit answers

These decisions will help determine whether you need additional State variables, without making your application unnecessarily complicated.

Me: For combination practice, I want: random. For regular division, I want: whole. Help me refine my State dataclass based on these decisions.

Great! Those choices actually make your State dataclass simpler because we can establish a consistent system for generating math problems.

Since combination practice will randomly select an operation and regular division will only use whole-number answers, here's how I would refine your design.

1. Recommended State dataclass

I would use the following:

None

```
@dataclassclass State:    # Navigation    current_page: str
# Math problem    number1: int    number2: int    operation:
str    correct_answer: int    # User interaction
user_answer: str    feedback: str    # Progress    score: int
questions_answered: int
```

I would keep this as your main State dataclass. You don't need a separate set of variables for each math page.

2. How each page uses the State

Here's how the different pages would interact with your variables.

Page	`current_page`	`operation`
Addition	`"addition"`	`"+"`
Subtraction	`"subtraction"`	`"-"`
Multiplication	`"multiplication"`	`"*"`
Division	`"division"`	`"/"`
Long Division	`"long_division"`	`"/"`
Combination	`"combination"`	Randomly selected

For combination, your application would randomly choose one of these four operations whenever it generates a new problem.

3. How problem generation would work

Your application will need to generate numbers based on the selected operation.

For example:

Example problems

8 + 4

Answer: 12

15 − 6

Answer: 9

7 × 3

Answer: 21

24 ÷ 6

Answer: 4

Illustrative examples of the four operations.

One important detail for division: generate the divisor and quotient first, then multiply them to get the dividend.

For example:

- *Divisor = 6*
- *Quotient = 4*
- *Dividend = 6 × 4 = 24*

This guarantees whole-number answers without needing complicated checks.

4. How I would handle scoring

Your scoring variables can be shared across every page.

For example:

Event	*Score*	*Questions answered*
Starting	*0*	*0*
Correct answer	*1*	*1*
Incorrect answer	*1*	*2*
Correct answer	*2*	*3*

You can calculate the percentage correct whenever you need it:

$$Percentage = \frac{score}{questionsanswered} \times 100$$

You don't need another State variable for the percentage.

5. A few things I would leave out for now

I would avoid adding these until you actually need them:

- *Separate addition, subtraction, multiplication, and division scores.*
- *Separate number variables for every page.*
- *A percentage variable.*
- *A list of every question the user has answered.*
- *Complicated long-division step tracking.*

These features could be added later if you have time.

6. One additional design decision

There's one thing I would like you to consider before finalizing your State dataclass.

How do you want users to interact with the math problems?

Answer input

Users type their answer into a text box.
Users select from multiple-choice answers.

Practice format

Continuous practice: Keep generating questions until the user decides to stop.
Ten questions per round, followed by a results screen.
Submit answers

These two decisions will help us determine whether your application needs any additional variables, particularly for tracking practice sessions.

Me: One of the requirements for the program is that at least one field of the state must be a list of class instances or a dictionary of class instances. How could I implement this into the state dataclass?

Yes! I would absolutely consider adding a timer to your application. In fact, it could be a really nice feature for your math practice website.

I would put the timer information in your State dataclass, but how you implement it depends on what kind of timer you want.

1. Decide what kind of timer you want

For your application, I see two possibilities:

- *Practice timer: Tracks how long the user spends practicing math problems.*
- *Question timer: Tracks how long the user takes to answer each individual question.*

My recommendation would be to start with a practice timer. It is simpler to implement and gives users a way to track their progress.

2. Add timer variables to State

You could add two fields:

```
None
@dataclassclass State:    # Navigation    current_page: str
# Current math problem    number1: int    number2: int
operation: str    correct_answer: int    # User interaction
user_answer: str    feedback: str    # Progress    score: int
questions_answered: int    # Problem history
problem_history: list[MathProblem] =
field(default_factory=list)    # Timer    timer_running: bool
elapsed_time: int
```

Here's what the timer variables mean:

Variable	Purpose
`timer_running`	*Determines whether the timer is currently active.*
`elapsed_time`	*Tracks the number of seconds elapsed.*

3. How would the timer work in Drafter?

Drafter supports timers, so you can use a timer component to periodically update your application's state.

The general process would be:

User clicks Start

timer_running = True

Timer updates elapsed_time

00:01 → 00:02 → 00:03

User clicks Stop

You could display the elapsed time on every math practice page.

4. An additional idea: Include time in MathProblem

If you decide you want to track how long users spend on individual questions, you could add another attribute to your `MathProblem` class:

```
None
@dataclassclass MathProblem:    number1: int    number2: int
operation: str    correct_answer: int    user_answer: str
was_correct: bool    time_taken: int
```

Now each problem could record how many seconds the user took to answer it.

This could eventually allow you to show users their average answer time.

5. My recommendation for your project

I would implement the features in this order:

1. *Add a practice timer to State.*
2. *Display the elapsed time on the practice pages.*
3. *Add Start and Stop buttons.*
4. *Once that works, consider tracking the time taken for individual questions.*

One important technical note: Don't rely on a timer callback being perfectly accurate. Browser tabs can become inactive or throttled. For a more reliable elapsed-time calculation, use the browser's time API to calculate the difference between the start time and current time.

One question: Do you want your timer to count up (showing how long someone has practiced), or count down (giving someone a time limit to complete their questions)?

That decision will help determine the best timer design for your State dataclass.

Me: I want a time limit. I want it to count down

Perfect! A countdown timer would work really well for your math practice application. You could let users complete as many problems as possible before time runs out.

I would make the timer part of your `State` dataclass.

1. Add countdown timer variables to State

I recommend adding these three fields:

```
None
# Timertime_remaining: inttimer_running: booltime_limit: int
```

Here's what each variable does:

Variable	Purpose
`time_remaining`	*Seconds remaining on the timer.*
`timer_running`	*Whether the countdown is active.*
`time_limit`	*The original time limit selected by the user.*

For example, if someone chooses a 60-second challenge:

- `time_limit = 60`
- `time_remaining = 60`
- `timer_running = True`

As time passes, `time_remaining` decreases.

2. How the countdown would work

Math Challenge Preview

TIME REMAINING

00:45

Timer running

12 × 4 = ?

YOUR ANSWER

48

Correct: 5

Questions: 7

Illustrative example of your future interface.

The basic sequence would be:

1. *User selects a time limit.*
2. *User clicks Start.*
3. *Timer begins counting down.*
4. *Each second, `time_remaining` decreases.*
5. *When `time_remaining` reaches zero, the challenge ends.*
6. *Display the user's final score.*

3. How to handle the timer in Drafter

Drafter supports timers, which can periodically trigger updates to your application.

One important programming concept: Don't make the countdown depend entirely on subtracting one every time a timer callback runs.

Browser timers aren't perfectly precise, especially when a browser tab is inactive.

A better approach is to store the time the challenge started and calculate the remaining time using the current time.

Conceptually:

None

```
time_remaining = time_limit - elapsed_seconds
```

You can use Python's `time` module to calculate elapsed seconds.

This also means that if the browser temporarily delays an update, your countdown can still display the correct remaining time.

4. My recommended State dataclass

Putting everything together, this is what I would envision:

None

```
@dataclassclass MathProblem:    number1: int    number2: int
operation: str    correct_answer: int    user_answer: str
was_correct: bool@dataclassclass State:    # Navigation
current_page: str    # Current problem    number1: int
number2: int    operation: str    correct_answer: int    #
User interaction    user_answer: str    feedback: str    #
Progress    score: int    questions_answered: int
problem_history: list[MathProblem] =
field(default_factory=list)    # Countdown timer
time_limit: int    time_remaining: int    timer_running: bool
start_time: float
```

`start_time` would store the starting timestamp so you can calculate how much time has passed.

5. One final design decision

How would you like users to select their time limit?

Choose a time limit

30 seconds – Quick challenge

60 seconds – Standard challenge

2 minutes – Extended challenge

Custom time limit – User enters their own time

Continue

I would recommend allowing users to select different time limits. It gives your application an additional useful feature without requiring a complicated backend or database.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.